

# ODFF: Operational Data Foundation Framework

Yoonseok Kang <sup>1</sup>  and Dongchul Park <sup>2,\*</sup> 

<sup>1</sup> Department of Convergence Security, Chung-Ang University, Seoul 06974, South Korea; kangcl1609@cau.ac.kr

<sup>2</sup> Department of Industrial Security, Chung-Ang University, Seoul 06974, South Korea; dongchul@cau.ac.kr

\* Correspondence: dongchul@cau.ac.kr

## Abstract

Smart manufacturing increasingly depends on continuous and usable operational data, yet in constrained small and medium-sized enterprise (SME) factories, the practical bottleneck often lies not in downstream analytics or AI applications but in the absence of an operationally viable data foundation. This study addresses that problem by proposing an *Operational Data Foundation Framework* for constrained SME manufacturing environments. The framework is derived from a lifecycle-based problem analysis and formalizes six core requirements: continuity, governance, diagnosability, operability, reprocessability, and evolvability. These requirements are further translated into design principles and operational invariants, thereby providing a structured basis for both design and assessment. To examine its field relevance, the framework was instantiated in a real SME deployment involving heterogeneous environmental sensors, a retrofitted legacy scale, LoRa-based field communication, edge-side persistence, cloud-side stream processing, and a governed canonical event model. The evaluation was conducted from the perspective of operational viability rather than benchmark performance alone. Requirement-based evidence showed that the implemented system preserved controlled semantics, supported segmented observability, maintained replayable historical records, and remained manageable under constrained deployment conditions. A representative field case further demonstrated the framework's practical value: LoRa reception instability was diagnosed through segment-wise discrepancy analysis and improved from approximately 33% to 95% packet reception after targeted intervention. The study contributes a field-grounded design logic for operational data systems that can support monitoring, traceability, analytics, alerting, and future extensions in digitally constrained manufacturing settings.

**Keywords:** smart manufacturing; SME digitalization; industrial data infrastructure; data governance; edge–cloud architecture

## 1. Introduction

Smart manufacturing is transforming manufacturing from a fixed and fragmented production environment into a connected operational system in which sensing, communication, computation, and decision making are tightly intertwined. Technologies such as the Internet of Things (IoT), cyber–physical systems (CPS), artificial intelligence and machine learning (AI/ML), real-time monitoring, predictive maintenance, and digital twins exemplify this shift. In this context, manufacturing systems can no longer be understood solely as physical production arrangements; they must also be understood as data-mediated operational systems.

Within this transition, data are no longer merely retrospective records. They have become a core operational resource that enables visibility, traceability, quality control,

Received:

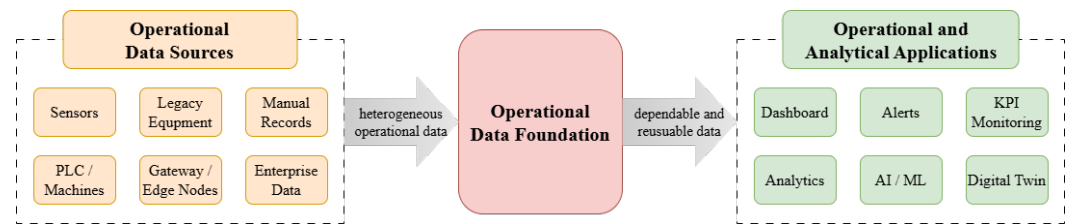
Revised:

Accepted:

Published:

**Copyright:** © 2026 by the authors.

Submitted to *Systems* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.



**Figure 1.** Position of the operational data foundation in a digitalized manufacturing system.

maintenance, optimization, and data-driven decision making. The practical advancement of smart manufacturing increasingly depends on whether operational data can be captured, interpreted, and reused in a stable and sustained manner across the shop floor. In this sense, smart manufacturing capability is closely tied to the continued availability of usable operational data.

At the same time, higher-level intelligent functions do not stand on their own. Predictive analytics, digital twins, and AI-driven optimization all presuppose an underlying operational layer that can continuously capture field data, preserve their meaning across system boundaries, and support downstream reuse. The central challenge, therefore, is not only what manufacturing systems can do more intelligently, but also which data conditions must first be maintained so that such intelligence becomes operationally viable.

The value of manufacturing data is not realized at the moment of generation alone. Rather, it emerges across an end-to-end flow that spans generation, acquisition, transmission, transformation, storage, and downstream use. For this reason, manufacturing data problems cannot be reduced to the performance of a single database, interface, or analytics component. They should instead be understood as operational systems problems distributed across the entire data lifecycle.

This challenge is intensified by the intrinsic heterogeneity of industrial data environments, where different devices, protocols, timestamps, machine states, manual records, and fragmented platforms coexist. Such conditions give rise to structural risks including missingness, duplication, delay, semantic inconsistency, timestamp ambiguity, and source-side variability. In manufacturing settings, these are not minor data imperfections. They directly affect traceability, weaken operational visibility, distort decision making, and ultimately reduce operational efficiency.

What is required, therefore, is not merely an ingestion infrastructure, but an operational layer capable of sustaining data continuity, governed semantics, durable persistence, observability, and downstream reusability. In this study, we refer to this layer as the *operational data foundation*. The core issue is thus not simply whether data can be collected, but whether they can be maintained as an operationally usable and reusable resource. From this perspective, the operational data foundation should not be understood as a mere conduit for data movement. Rather, it provides the basis on which heterogeneous field data can be stabilized, governed, and made dependable for sustained operational and analytical use. As illustrated in Figure 1, it occupies the intermediate layer between heterogeneous operational data sources and higher-level operational and analytical applications.

Although building dependable manufacturing data systems is challenging in general, its foundational requirements become especially visible in constrained small and medium-sized enterprise (SME) factories. These environments rarely offer the conditions often assumed in digitally mature smart-manufacturing settings, such as standardized interfaces, stable internal connectivity, dedicated technical support, and integrated data infrastructure. Instead, they more often combine legacy equipment, fragmented systems, mixed communication technologies, manual records, limited technical staffing, and strong cost sensitivity. For this reason, SME factories should not be regarded merely as smaller-scale plants, but

as critical design settings in which the hidden assumptions of smart manufacturing data systems are exposed most clearly.

Under such conditions, the practical bottleneck of digitalization lies not simply in deploying more sensors, collecting more data, or adding downstream applications. Rather, it lies in whether operational data can be captured with continuity, stabilized with interpretable meaning, and sustained as a usable resource despite fragmentation, instability, and limited operational slack. In SME manufacturing, the central difficulty is therefore not merely data scarcity, but the fragility of the operational data lifecycle through which field data must become dependable for ongoing operational use.

This implies that the primary bottleneck of smart manufacturing in constrained SME factories lies not in downstream intelligence, but in establishing an operationally viable data foundation. Accordingly, this study addresses the following problem: *how should an operational data foundation be designed, and what must it guarantee, in order to make smart manufacturing practically viable in constrained SME factories?*

Existing research has addressed manufacturing data systems from several complementary directions. Smart manufacturing studies have primarily focused on higher-level applications and intelligent capabilities, including real-time monitoring, predictive maintenance, digital twins, and adaptive production support. In parallel, research on industrial data infrastructures has examined pipelines, integration architectures, edge-cloud coordination, semantic consistency, governance, and data quality. Related lifecycle-oriented work has emphasized continuity, traceability, reusable linkage, and the preservation of context across stages, while studies on SME digitalization have clarified the realities of constrained deployment, including legacy equipment, fragmented systems, limited staffing, and strong cost sensitivity.

However, these contributions remain only partially integrated from the perspective of operational viability under constrained deployment conditions. Prior work has often addressed applications, architectural components, lifecycle properties, or SME deployment constraints as related but separate concerns, rather than formalizing what a manufacturing data foundation must guarantee in order to remain operationally usable in practice. As a result, there is still limited systems-level articulation of how these interrelated concerns can be brought together into a coherent design basis for constrained SME manufacturing environments.

To address this gap, this study focuses on the foundational layer that smart manufacturing applications presuppose and formalizes it in terms of operational requirements, design logic, and verification basis. More specifically, this study develops an *Operational Data Foundation Framework* for constrained SME factories and examines it through field-grounded implementation and evidence.

Based on this motivation, the study is guided by the following research questions:

- **RQ1.** What operational risks emerge when smart manufacturing data pipelines are deployed under constrained SME factory conditions?
- **RQ2.** What core requirements must an operational data foundation satisfy in such conditions, and how can they be formalized into design principles and operational invariants?
- **RQ3.** How can the proposed framework be instantiated in a real SME manufacturing setting, and how can its practical value be examined through field evidence?

To answer these questions, this study first organizes a general lifecycle model of smart manufacturing data pipelines as an analytical reference. It then maps constrained SME factory conditions onto that lifecycle in order to identify the key operational risks that hinder practical deployment. In response to these risks, the study formalizes the Operational Data Foundation Framework in terms of core requirements, design principles, and operational

invariants. Finally, it instantiates the framework in a real SME manufacturing setting and examines its practical value through requirement-based evidence and representative field observations.

The primary contributions of this research are as follows:

1. Redefining the actual bottleneck of smart manufacturing in constrained SME factories not as a downstream application problem, but as an operational data foundation problem.
2. Deriving the core operational risks that emerge when a general manufacturing data lifecycle is subjected to constrained SME factory conditions.
3. Formalizing the Operational Data Foundation Framework (ODFF), structured around core requirements, design principles, and operational invariants.
4. Validating the framework's field-grounded operational relevance through real-world implementation in an SME manufacturing environment.

The remainder of this paper is organized as follows. Section 2 reviews the related literature. Section 3 analyzes the general data pipeline lifecycle, SME-specific constraints, and the resulting operational risks. Section 4 presents the proposed Operational Data Foundation Framework. Section 5 describes the field implementation in an SME manufacturing setting. Section 6 evaluates the framework using requirement-based evidence and field observations. Finally, Section 7 concludes the paper.

## 2. Related Work

### 2.1. Smart Manufacturing Applications

Smart manufacturing research has developed mainly around higher-level applications and intelligent capabilities, including industrial IoT, cyber-physical systems, digital twins, real-time monitoring, predictive maintenance, and AI/ML-enabled decision making [1–4]. Across these streams, the dominant goal has been to make manufacturing systems more connected, adaptive, and data-driven through improved visibility, prediction, and control [1,2,5].

Despite their diversity, these applications share a common prerequisite: continuous or near-real-time operational data from the shop floor [6–8]. Studies on digital twins and real-time manufacturing intelligence repeatedly show that higher-level functions depend on accurate, timely, and interpretable data aligned with physical system states [6,9,10]. Yet this literature is generally more explicit about what smart manufacturing applications can optimize, predict, or automate than about what kind of data basis must first be sustained for those functions to remain operationally viable in practice [1,6,11].

### 2.2. Industrial Data Infrastructures

A substantial body of research has examined the industrial data basis through manufacturing data pipelines, integration architectures, industrial data management systems, and edge-cloud infrastructures [12–14]. This literature makes clear that manufacturing data handling is not a single storage problem, but a multi-layer systems problem spanning acquisition, communication, processing, storage, and downstream delivery across heterogeneous sources [12,15].

Within this stream, two themes are especially prominent. First, many studies address heterogeneous integration and interoperability across legacy equipment, devices, protocols, platforms, and enterprise systems [16–18]. Second, related work has emphasized semantics, context, and governance, arguing that industrial data must remain interpretable, trustworthy, and reusable rather than merely transferable [19–21]. These contributions have significantly advanced the understanding of how industrial data infrastructures can be connected, organized, and managed. However, they remain more focused on compo-

nents, architectures, and enabling techniques than on explicitly formalizing what such infrastructures must guarantee as an operational foundation under deployment constraints [12,13,20].

### 2.3. Lifecycle Connectivity

This view has been developed most clearly through the concept of the digital thread. Recent reviews and implementation studies define the digital thread as a means of linking lifecycle information across heterogeneous artifacts and stages, thereby enabling traceability, cross-stage visibility, and reuse beyond isolated system silos [22–25]. Related work on contextualization, standards-based linkage, and knowledge-graph integration similarly shows that lifecycle usefulness depends not only on storing data, but on preserving associativity, context, and interpretability for later reuse [26–28]. However, this literature has focused mainly on linkage and traceability, leaving relatively less explicit discussion of how such continuity and reuse can be sustained operationally under constrained deployment conditions.

The concept of the digital thread has become the clearest expression of this perspective. Recent reviews and implementation studies define the digital thread as a means of linking information across heterogeneous lifecycle artifacts and stages so that traceability, cross-stage visibility, and lifecycle-wide reuse become possible beyond fragmented system silos [22–25]. Related studies on contextualization, standards-based linkage, and knowledge-graph integration likewise show that lifecycle usefulness depends not merely on storing records, but on preserving associativity, context, and interpretability for later reuse [26–28]. At the same time, this literature has concentrated mainly on linkage and traceability, leaving relatively less explicit discussion of how such continuity and reuse can be sustained operationally under constrained deployment conditions.

### 2.4. SME Deployment

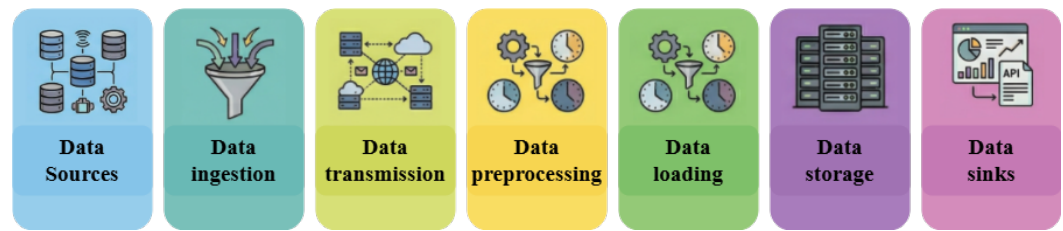
The SME literature makes those deployment constraints explicit. Studies on smart manufacturing adoption in SMEs consistently show that these environments differ substantially from idealized smart factory assumptions: legacy equipment, fragmented systems, uneven interfaces, limited digital infrastructure, restricted investment capacity, and uneven technical expertise are common [29–32]. In this sense, SMEs should not be viewed simply as smaller large factories, but as constrained deployment settings in which the assumptions of digitally mature architectures are most directly challenged.

This constraint is both technical and operational. Brownfield dependence often forces SMEs to rely on retrofitting, add-on sensing, gateway-based integration, and low-cost connectivity rather than wholesale replacement [33–36]. At the same time, recurrent barriers include limited staffing, weak digital skills, uncertain return on investment, high implementation cost, and maintainability burden [37–40]. Accordingly, SME-oriented studies increasingly advocate phased, modular, and capability-building adoption paths [32,41,42]. Taken together, these studies show that the unresolved problem is not how to deploy smart manufacturing application, infrastructure component, or lifecycle linkage mechanism, but how to sustain a data foundation that remains operationally viable under constrained real-world conditions.

## 3. Problem Analysis

This section analyzes the data basis of smart manufacturing as an operational pipeline and examines how constrained SME factory conditions impose structural fragility across that pipeline. Rather than merely describing deployment difficulty, it identifies how lifecycle-wide constraints converge into a small set of recurring operational risks. In this





**Figure 2.** Enter Caption

way, the section defines the problem structure of operational data foundations under constrained deployment conditions.

### 3.1. General Data Pipeline Lifecycle

A common analytical reference is required before constrained SME conditions and their resulting risks can be examined in a structured way. In smart manufacturing, data do not become operationally useful at the moment of generation alone. Their practical value emerges only through a broader path in which field observations are accepted into a pipeline, transferred across infrastructure layers, transformed into usable forms, preserved, and connected to downstream operational use. For this reason, this study adopts a *general data pipeline lifecycle* as an analytical frame for problem interpretation rather than as a normative system architecture.

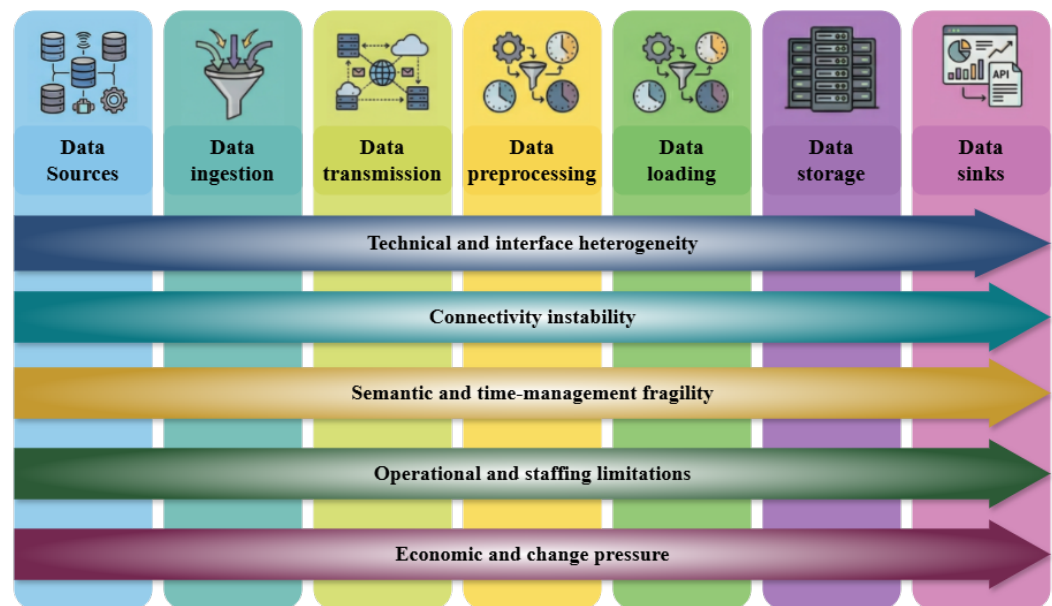
The lifecycle used in this study is shown in Figure 2. It is adapted from generic data pipeline literature that treats data systems as multi-stage processing structures with recurring issues around ingestion, integration, cleaning, transformation, loading, and pipeline-wide quality control [43]. In the smart manufacturing context, this general view is organized as follows:

- **Data sources:** the points at which operational data originate, including sensors, equipment states, manual records, and external systems.
- **Data ingestion:** the intake of source data into the pipeline.
- **Data transmission:** the movement of acquired data across gateway, edge, server, or cloud boundaries toward the next processing point.
- **Data preprocessing:** the cleaning, decoding, normalization, mapping, and transformation required to make raw inputs usable downstream.
- **Data loading:** the controlled insertion of processed records into the target operational layer or storage boundary.
- **Data storage:** the durable preservation of records for later access, recovery, reinterpretation, and reuse.
- **Data sinks:** the downstream endpoints that consume stored data, including dashboards, alerts, reporting, analytics, and decision support.

This lifecycle is analytically useful because it makes visible the full path through which manufacturing data become operationally usable. At the same time, it is not a design answer on its own. It describes how data move, but not what must be preserved so that the lifecycle remains operationally viable under constrained conditions. It is therefore used in this study as an analytical reference for the constraint and risk analysis that follows.

### 3.2. SME Constraints

Constrained SME factories should not be treated simply as smaller versions of digitally mature plants. Prior research shows that their starting conditions are structurally different: legacy equipment, fragmented systems, uneven interfaces, restricted digital infrastructure, limited internal expertise, and strong investment sensitivity are common, whereas standardized connectivity and unified data access are often absent [29–32,44]. SME constraints



**Figure 3.** Enter Caption

are therefore better understood not as isolated implementation difficulties, but as recurring design conditions acting across the general data pipeline lifecycle. Figure 3 highlights this point by showing how the major constraint types propagate across source capture, transfer, transformation, storage, and downstream use rather than remaining confined to a single stage.

Heterogeneity begins at the source side. In SME factories, brownfield machinery, retrofit-based extension, add-on sensing, and gateway-mediated access are common, so inconsistency is already present at the level of data sources and ingestion rather than appearing only later in integration [33–36]. The challenge is not merely collecting values, but incorporating diverse formats, units, reporting intervals, and source identities into a coherent operational structure. Source-side variability is thus not a peripheral inconvenience, but the starting condition from which later coordination and interpretation problems arise.

Transmission is often fragile in constrained SME deployments. In constrained SME deployments, field data may depend on low-cost wireless communication, battery-powered sensing, and gateway- or edge-dependent relay paths rather than on stable plant-wide infrastructure [45–47]. The ingestion–transmission path therefore becomes a distinct surface of instability. What matters operationally is not simply whether a value was observed once, but whether the record can remain sufficiently continuous and recoverable despite interruption, delay, and relay dependence.

Operational meaning must remain stable across the pipeline. Manufacturing data are operationally usable only when source identity, metric semantics, unit interpretation, and time semantics stay controlled across preprocessing, loading, and storage. Yet constrained environments frequently rely on heterogeneous preprocessing logic, ad hoc mappings, incomplete context, and weak contract discipline, making semantic drift and traceability loss structurally likely [17,19–21,48]. Once such instability becomes embedded in stored records, downstream analytics and dashboards may continue to function while resting on an increasingly fragile semantic basis.

These technical difficulties are intensified by operational reality. SME studies repeatedly report limited staffing, uneven digital skills, manual troubleshooting, high maintenance burden, and weak strategic capacity [37,39,40,49]. At the same time, SME digitalization rarely proceeds as a one-shot transformation; it unfolds through phased rollout, retrofit-based extension, KPI revision, and incremental onboarding of new sources and pro-

cesses [41,42,50,51]. As a result, the practical challenge is not only whether a system can be built, but whether it can remain diagnosable, manageable, and adaptable as requirements change.

Taken together, these observations show that the core problem in constrained SME factories is not simply that deployment is difficult. Rather, the assumptions of the general data pipeline lifecycle are systematically destabilized across the lifecycle itself: records may lose continuity, meanings may drift, failures may become hard to localize, operational burden may grow, historical data may lose replay value, and later extension may require disproportionate rework. The next subsection translates these recurring forms of fragility into the language of operational risk.

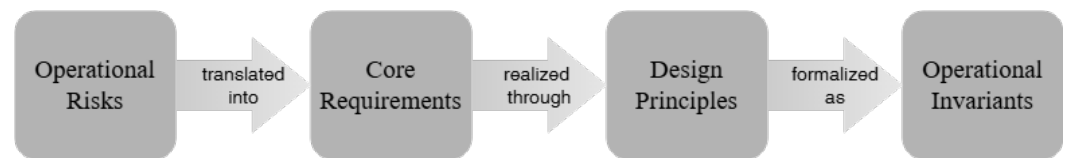
### 3.3. Operational Risks

The SME constraints discussed above do not remain as isolated deployment difficulties. When mapped onto the general data pipeline lifecycle, they appear as recurring operational risks that determine whether manufacturing data can remain usable in practice. The issue is therefore not only how data move through the pipeline, but what must be preserved so that the resulting record remains dependable under constrained real-world conditions.

- RSK1. Continuity risk.** Operational records may become interrupted, fragmented, or partially lost as data move from source capture to storage. In constrained manufacturing settings, the essential requirement is not one-time collection success, but the preservation of a gap-minimized and recoverable record despite source instability, transmission disruption, or delayed recovery.
- RSK2. Governance risk.** Data may remain available while losing stable meaning. This occurs when source identity, metric semantics, timestamp interpretation, or transformation logic are not kept consistent across collection, preprocessing, loading, and storage. Under such conditions, downstream analysis may continue to operate, but on data whose meaning is no longer reliably governed.
- RSK3. Diagnosability risk.** When failure occurs, it may be difficult to determine where in the lifecycle the problem emerged and what kind of failure it is. Weak monitoring, poor stage separation, and limited observability turn degradation into a black-box symptom, making it difficult to distinguish transmission loss, semantic corruption, storage inconsistency, or downstream mismatch in a structurally interpretable way.
- RSK4. Operability risk.** A system may function technically while becoming too costly or complex to sustain in everyday practice. Tight coupling, heavy maintenance burden, unclear ownership, and non-modular structures raise the effort required to manage, modify, and repair the system, making long-term operation difficult under constrained SME staffing and budget conditions.
- RSK5. Reprocessability risk.** Historical data may cease to be usable when rules, KPIs, schemas, or interpretation logic change. If raw preservation is weak, preprocessing is effectively irreversible, or lineage is incomplete, past records cannot be reliably replayed, recomputed, or reinterpreted. In that case, the data foundation loses one of its most important operational values: the ability to support correction and renewed interpretation over time.
- RSK6. Evolvability risk.** New sensors, processes, lines, factories, or downstream requirements may become difficult to incorporate without disproportionate redesign cost or structural disruption. Since SME digitalization usually proceeds through incremental onboarding rather than one-shot integration, a data foundation that cannot absorb change without breaking governance and operational control cannot serve as a durable basis for growth.

Taken together, these risks show that a smart manufacturing data pipeline in constrained SME factories cannot be treated as a simple ingestion flow. The problem is not a





**Figure 4.** Enter Caption

collection of isolated device, network, or software issues, but a structural fragility of the lifecycle through which operational data become usable. What is required, therefore, is a foundation that can explicitly preserve continuity, governance, diagnosability, operability, reprocessability, and evolvability under constrained deployment conditions. The next section presents the *Operational Data Foundation Framework* as a structured response to these risks.

## 4. Operational Data Foundation Framework

### 4.1. Framework Overview

Section 3 showed that, in constrained SME factory environments, smart manufacturing data pipelines are exposed to recurring risks of continuity loss, semantic instability, weak diagnosability, operational burden, limited reprocessability, and poor evolvability. These risks indicate that the data layer cannot be treated merely as a transmission path or collection mechanism. What is needed instead is an explicit framework that defines what an operational data foundation must guarantee in order to remain viable under constrained deployment conditions.

To address this need, this study proposes the *Operational Data Foundation Framework*. As shown in Figure 4, the framework is organized as a risk-to-design logic: *operational risks* are translated into *core requirements*; these requirements are realized through *design principles*; and their realized state is assessed through *operational invariants*. In this way, the framework converts the problem structure identified in Section 3 into an explicit and assessable basis for the design and evaluation of an operational data foundation.

### 4.2. Core Requirements

The framework begins by defining the *core requirements* of an operational data foundation. These requirements are not desirable properties in a general sense, but the minimum lifecycle-wide guarantees needed for operational viability under constrained SME factory conditions. They provide the basis for the design principles and operational invariants developed in the following subsections.

Six core requirements are defined.

- R1. Continuity.** Data continuity should be preserved as far as practicable despite field disruption and connectivity instability. The essential requirement is not momentary acquisition success, but the minimization of gaps and the preservation of recoverable continuity so that the operational record remains usable even when intermediate segments are unstable.
- R2. Governance.** Source identity, metric semantics, and time semantics should remain stable across collection, normalization, and storage. Data are not operationally usable merely because they exist; they must retain controlled meaning and interpretable context throughout the pipeline.
- R3. Diagnosability.** Failures should be observable as segment-level discrepancies rather than only as end-to-end symptoms. The requirement is not simply that problems become visible after damage appears downstream, but that the lifecycle location and mode of failure can be identified in a structurally interpretable way.

- R4. Operability.** The system should remain maintainable and manageable under constrained staffing and budget conditions. An operational data foundation must therefore be sustainable in day-to-day practice without imposing structurally excessive maintenance burden or operational complexity.
- R5. Reprocessability.** Historical data should remain reprocessable for rule changes, KPI revisions, correction, and recovery. Manufacturing data should not be treated as one-time consumables, but as records that can be read again, recalculated, and reinterpreted when operational needs evolve.
- R6. Evolvability.** The foundation should accommodate new sources, processes, sites, and applications without breaking the existing structure. Since SME digitalization typically proceeds through incremental onboarding rather than one-time integration, the data foundation must remain structurally open to change and extension.

#### 4.3. Design Principles

The core requirements defined above specify what the operational data foundation must guarantee, but not how those guarantees should be realized in design. This subsection therefore presents the key *design principles* that translate those requirements into a workable design logic for constrained SME factory conditions. Rather than prescribing specific technologies or implementation choices, these principles define the structural directions needed to sustain the required operational guarantees.

- P1. Continuity responsibility should be placed close to the source.** If continuity depends primarily on downstream transmission or cloud reachability, it becomes fragile under field disruption and connectivity instability. For this reason, the first responsibility for preserving continuity should be located as close to the source as possible through source-near capture, upstream buffering or persistence, and recovery-oriented ingestion logic.
- P2. Raw events should be preserved before abstraction.** If only transformed outputs are retained, later replay, reinterpretation, and recovery become structurally limited. The preservation of raw events before irreversible abstraction is therefore a fundamental design principle, since rule changes, KPI revision, and correction all depend on access to preserved pre-abstraction records.
- P3. Semantics should be contract-defined.** If source identity, metric naming, timestamp semantics, unit definition, and transformation logic are left to ad hoc implementation, semantic inconsistency accumulates across the pipeline. The design must therefore define semantics explicitly through controlled contracts, schema discipline, version-aware rules, and clear identity and time conventions.
- P4. Observability should follow pipeline segments.** Diagnosability cannot be secured through black-box end-to-end monitoring alone, because operationally meaningful failures must be localizable to specific lifecycle segments. Observability should therefore be structured along the pipeline itself through segment-wise checkpoints, stage-level indicators, and explicit discrepancy visibility.
- P5. Responsibilities should be separated by function.** If acquisition, normalization, storage, and downstream use are tightly coupled, both maintenance burden and change cost rise rapidly. Functional responsibilities should therefore be separated so that components remain structurally independent, interfaces remain replaceable, and modifications in one part do not unnecessarily destabilize the whole.
- P6. Operations should remain lightweight, modular, and open.** In constrained SME environments, heavy operational structures, costly maintenance models, and vendor dependence undermine long-term viability. The design should therefore remain lightweight in deployment, modular in composition, manageable in maintenance

scope, and open to replacement and extension. Where appropriate, open and widely supported components, including open-source tools, can help reduce lock-in and maintenance burden.

- P7. Change should be treated as a first-class assumption.** Changes in KPIs, schemas, rules, source onboarding, and deployment scope are not exceptional events but normal operating conditions in SME digitalization. The foundation should therefore be designed from the outset to tolerate and organize change through version-aware structures, replay support, incremental onboarding discipline, and change-tolerant interfaces.

#### 4.4. Operational Invariants

The core requirements and design principles defined above specify what the operational data foundation must guarantee and how those guarantees should be realized. This subsection completes the framework by defining the *operational invariants*: the persistent conditions that should hold if the foundation is functioning as intended.

- I1. Durable Capture Invariant.** An accepted event should obtain durable persistence before downstream failure can erase it. Continuity should therefore be judged from the point of first durable capture rather than from downstream availability alone.
- I2. Traceability Invariant.** Every canonical or stored record should remain traceable to its source identity and operational context. Downstream data should not become detached from the source, metadata, and governing semantics that give it meaning.
- I3. Deterministic Normalization Invariant.** The same raw event under the same contract and version conditions should always produce the same canonical output. Normalization should function as a repeatable and controlled semantic transformation rather than as ad hoc interpretation.
- I4. Segment-Localizable Failure Invariant.** A failure should become visible as at least one discrepancy within a recognizable lifecycle segment. It should be possible to attribute the problem to a specific segment such as source, transmission, preprocessing, loading, storage, or sink rather than observing it only as a downstream symptom.
- I5. Replayability Invariant.** Preserved raw data should remain reprocessible for rule changes, KPI revision, correction, and recovery. Historical data should therefore remain usable for later reinterpretation and recomputation rather than serving only as passive archive material.
- I6. Modular Replacement Invariant.** Replacing a specific component should not break the semantic chain of the overall foundation. Changes in storage, parsing, transport, or downstream components should not destroy source identity, metric meaning, or replay context across the system.
- I7. Onboarding Consistency Invariant.** A new source should be incorporable into the existing structure without breaking the prevailing governance discipline. Incremental onboarding should proceed through the same contract, schema, naming, and validation discipline rather than through an accumulation of ad hoc exceptions.

These invariants define the persistent conditions by which the framework can be assessed in practice. If requirements declare the guarantees to be achieved and principles define the design logic needed to achieve them, invariants provide the basis for evaluating whether those guarantees remain structurally preserved.

#### 4.5. Framework Synthesis

This subsection synthesizes the framework by reconnecting it to the problem space analyzed in Section 3. Section 3 showed how constrained SME factory conditions destabilize the general data pipeline lifecycle and generate recurring risks of continuity, governance,

diagnosability, operability, reprocessability, and evolvability. Sections 4.2–4.4 translated those risks into core requirements, design principles, and operational invariants. The purpose here is to clarify how these layers are systematically linked to the original problem structure.

Table 1 provides this first synthesis. It shows that constrained SME conditions are not merely a list of deployment difficulties, but recurring operational design problems that can be translated in a structured way. Each major constraint category gives rise to one or more operational risks; each risk is then expressed as a core requirement, translated into one or more design principles, and made assessable through corresponding invariants. The table thus shows how the framework is grounded in recurring structural problems in constrained SME manufacturing.

**Table 1.** From SME Constraints to Operational Design Logic.

| SME Constraint                                  | Operational Risk               | Requirement           | Design Principle                                                  | Invariant                                           |
|-------------------------------------------------|--------------------------------|-----------------------|-------------------------------------------------------------------|-----------------------------------------------------|
| Heterogeneous devices and fragmented interfaces | RSK2 (Governance risk)         | R2 (Governance)       | P3 (Semantics must be contract-defined)                           | I2 (Traceability), I3 (Deterministic normalization) |
| Add-on sources and partial visibility           | RSK2, RSK6 (Evolvability risk) | R2, R6 (Evolvability) | P3, P7 (Change as a first-class assumption)                       | I2, I7 (Onboarding consistency)                     |
| Unstable connectivity and constrained sensing   | RSK1 (Continuity risk)         | R1 (Continuity)       | P1 (Continuity responsibility must be placed close to the source) | I1 (Durable capture)                                |
| Weak cross-segment observability                | RSK3 (Diagnosability risk)     | R3 (Diagnosability)   | P4 (Observability must follow pipeline segments)                  | I4 (Segment-localizable failure)                    |
| Limited staff and high maintenance burden       | RSK4 (Operability risk)        | R4 (Operability)      | P6 (Operations must remain lightweight, modular, and open)        | I6 (Modular replacement)                            |
| Weak raw-data preservation                      | RSK5 (Reprocessability risk)   | R5 (Reprocessability) | P2 (Raw events must be preserved before abstraction)              | I5 (Replayability)                                  |
| Tight functional coupling across the pipeline   | RSK3, RSK4, RSK6               | R3, R4, R6            | P5 (Responsibilities must be separated by function)               | I4, I6                                              |
| Frequent KPI, rule, schema, and source changes  | RSK5, RSK6                     | R5, R6                | P7                                                                | I5, I7                                              |
| Budget sensitivity and lock-in concerns         | RSK4, RSK6                     | R4, R6                | P6                                                                | I6, I7                                              |

One particularly important translation highlighted in Table 1 is the movement from semantic and interface heterogeneity to governance logic. In constrained SME factories, heterogeneous devices, mixed protocols, fragmented interfaces, add-on sensing, and manual records do not merely create diverse formats; they destabilize source identity, metric meaning, unit interpretation, and timestamp semantics. The issue is therefore not simply one of integration, but one of governance. For this reason, the framework connects this problem to *R2 Governance*, *P3 contract-defined semantics*, and the assessable conditions of *I2 Traceability* and *I3 Deterministic Normalization*. This translation makes clear that manufacturing data become operationally reliable only when meaning and origin remain stably governed.

A second important translation concerns the movement from change pressure to reprocessability and evolvability logic. SME digitalization rarely proceeds as a one-time integration event; instead, KPI definitions evolve, schemas change, new sensors and lines are added, and operational rules are revised over time. When such change is not treated as a normal operating condition, transformed-only persistence and weak lineage make past data difficult to reinterpret and new sources difficult to onboard without exception-heavy modification. The framework therefore connects this problem to *R5 Reprocessability* and *R6 Evolvability*, supported by *P7 change as a first-class assumption* and assessed through *I5 Replayability* and *I7 Onboarding Consistency*. This shows that the framework is designed not only for present-time correctness, but also for preserving operational coherence under future change.

Table 2 provides a second synthesis by showing how the framework adds operational meaning to the general data pipeline lifecycle. Under constrained SME conditions, each lifecycle stage must contribute not only to data movement and processing, but also to the guarantees defined by the operational data foundation. The table makes this added meaning explicit by connecting each stage to the relevant risks, requirements, design principles, and invariants. In this way, the general lifecycle is retained as an analytical reference, while its role is sharpened for constrained manufacturing conditions.

One especially revealing example is the reinterpretation of the transmission stage. In generic pipeline discussions, transmission is often treated as a routine internal link between ingestion and later processing. In constrained SME factory conditions, however, the path from field devices to gateways, edge systems, servers, and cloud systems is often exposed to intermittent connectivity, limited bandwidth, battery constraints, delayed recovery, and relay dependence. Transmission therefore becomes a distinct operational segment in which continuity and diagnosability are directly tested. For this reason, Table 2 assigns the transmission stage the added meanings of recoverable transfer and segment-aware discrepancy visibility, linking it to *R1 Continuity*, *R3 Diagnosability*, *P1 source-near continuity*, *P4 segment-based observability*, *I1 Durable Capture*, and *I4 Segment-Localizable Failure*.

Taken together, Table 1 translates constrained SME conditions into operational design logic, while Table 2 shows how that logic refines the interpretation of the general data pipeline lifecycle under real manufacturing constraints. The result is a single analytical structure that connects problem space, framework logic, and lifecycle interpretation. This synthesis also provides the basis for reading the field implementation in Section 5 and the evidence assessment in Section 6.



**Table 2.** Mapping the General Lifecycle to the Operational Foundation Logic.

| General Lifecycle Stage                        | Added Operational Meaning                         | Related Risks | Requirements | Principles | Invariants |
|------------------------------------------------|---------------------------------------------------|---------------|--------------|------------|------------|
| Data sources                                   | Stable source identity                            | RSK2, RSK6    | R2, R6       | P3, P7     | I2, I7     |
| Data ingestion                                 | Source-near continuity and intake control         | RSK1, RSK2    | R1, R2       | P1, P3     | I1, I2     |
| Data transmission                              | Recoverable transfer and visible discrepancies    | RSK1, RSK3    | R1, R3       | P1, P4     | I1, I4     |
| Data preprocessing                             | Contract-defined and deterministic transformation | RSK2, RSK6    | R2, R6       | P3, P7     | I2, I3, I7 |
| Data loading                                   | Controlled persistence with replay context        | RSK5, RSK6    | R5, R6       | P2, P7     | I5, I7     |
| Data storage                                   | Replay-ready and replaceable persistence          | RSK4, RSK5    | R4, R5       | P2, P6     | I5, I6     |
| Data sinks                                     | Governed downstream reuse                         | RSK2, RSK5    | R2, R5       | P3, P7     | I2, I5     |
| Monitoring and management across the lifecycle | Lifecycle-wide observability and maintainability  | RSK3, RSK4    | R3, R4       | P4, P6     | I4, I6     |

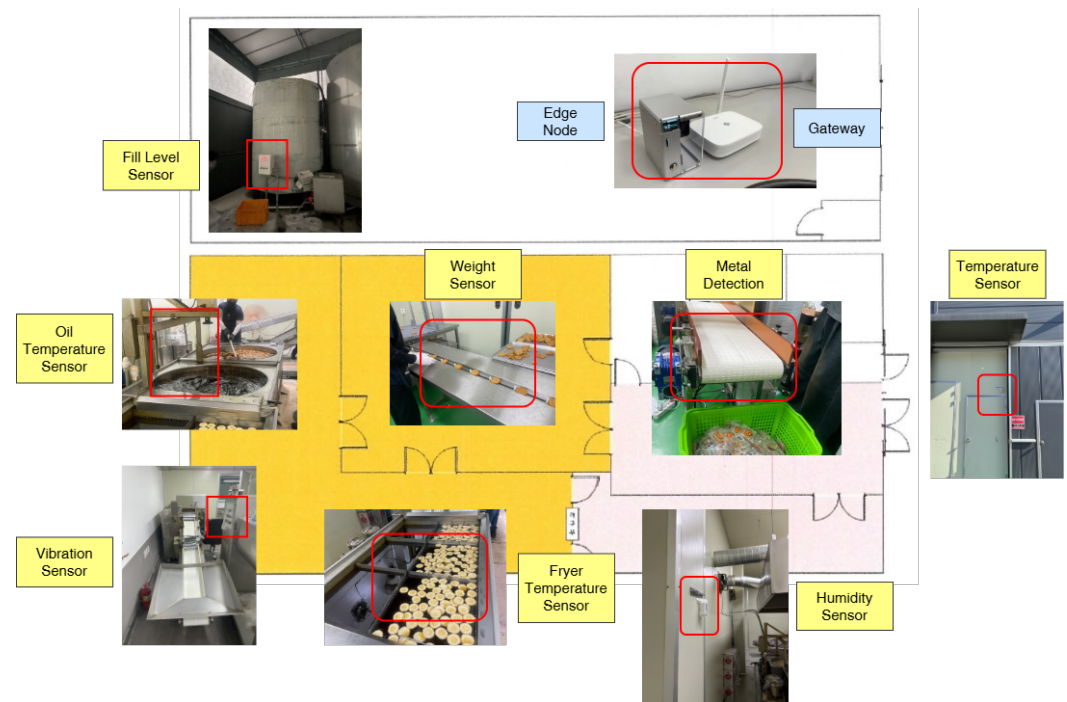
**5. Field Implementation**

This section describes how the proposed framework was instantiated in a real SME manufacturing setting. The implementation is presented not as a generic IoT deployment, but as a field realization of an operational data foundation under constrained conditions. The focus is therefore placed on how heterogeneous sources were incorporated, how continuity was secured close to the source, how semantics were stabilized through the processing and storage path, and how the resulting data layer was connected to operational visibility and intervention.

*5.1. Plant Context and Sensing Objectives*

The empirical setting of this study was the Idang Food yakgwa plant, one of several OEM factories managed by ONGOING, an F&B company operating across multiple production sites. The deployment was therefore designed not as an isolated factory pilot, but as the first field implementation of a broader architecture in which operational data from multiple factories could be collected locally and managed through a shared cloud-side structure. In this sense, Idang Food was selected not only as a practical implementation site, but also as a representative case through which this study examines how an operational data foundation can be established under constrained SME manufacturing conditions.

The plant’s production flow included raw-material receiving, weighing, mixing, forming, frying, coating, cooling, inner packaging, detection, outer packaging, and storage. Yet the main challenge of the site did not lie simply in the number of stages. Operationally relevant states were distributed across different rooms, devices, and tasks, while production



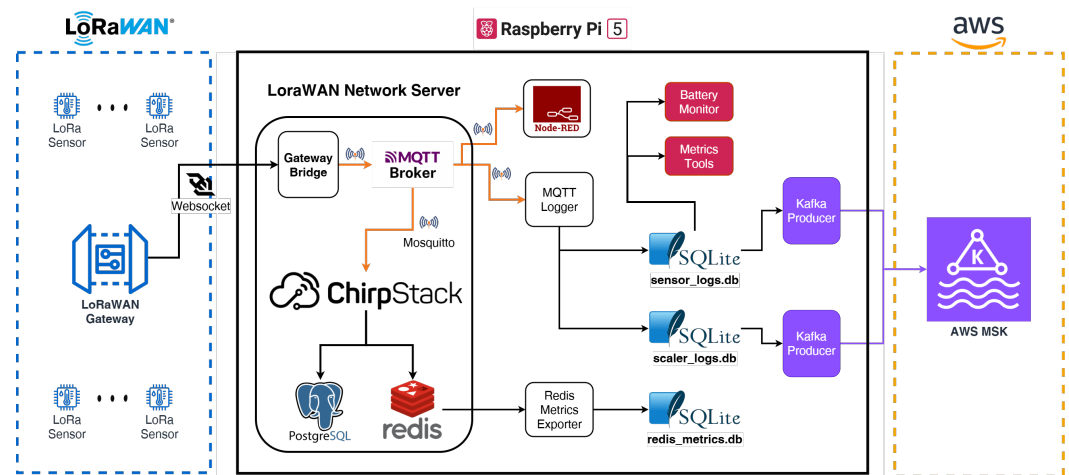
**Figure 5.** Enter Caption

and quality management depended heavily on manual records and worker judgement. Process conditions were not accumulated as connected records, weighing-related checks remained strongly operator-dependent, and environmental histories across process and storage zones were difficult to preserve and compare over time. As a result, when abnormal conditions or quality issues emerged, the plant did not lack observations in an absolute sense; rather, it lacked continuous and reusable operational records that could support traceability and later interpretation.

This made the site suitable for the present study. Idang Food reflected several conditions that commonly characterize constrained SME factories: heterogeneous and partly non-networked sources, fragmented process visibility, limited internal connectivity, and restricted operational slack for day-to-day technical management. Under such conditions, an operational data foundation could not be assumed to arise simply from downstream software or cloud connectivity. It first had to be built from the field upward by making process states recordable, collectable, and sustainable as data under actual production constraints.

The field deployment should be understood in this light. The study did not begin from an already prepared digital source environment, but from the practical requirement to establish one inside the factory through on-site sensing and edge-side installation. Figure 5 presents this deployment context by showing the production environment together with representative sensing and edge elements installed in the field. The significance of the case, however, lies less in the installation itself than in what that installation enabled: a concrete demonstration of how a constrained SME factory can begin to acquire the operational basis required for continuity, traceability, and later cloud-side integration.

Accordingly, the role of Idang Food in this study was twofold. At the factory level, it provided a realistic constrained environment in which fragmented sources, manual practices, and brownfield conditions had to be handled in practice. At the system level, it served as the first implementation site of an architecture intended to support factory-to-cloud data flow and centralized multi-factory management. For this reason, the next subsection begins with field data acquisition and the edge architecture, because the first requirement of the



**Figure 6.** Enter Caption

operational data foundation was to explain how field data were physically captured and operationally secured at the factory before they were forwarded into the broader cloud-side system.

## 5.2. Field Data Acquisition and Edge Stack

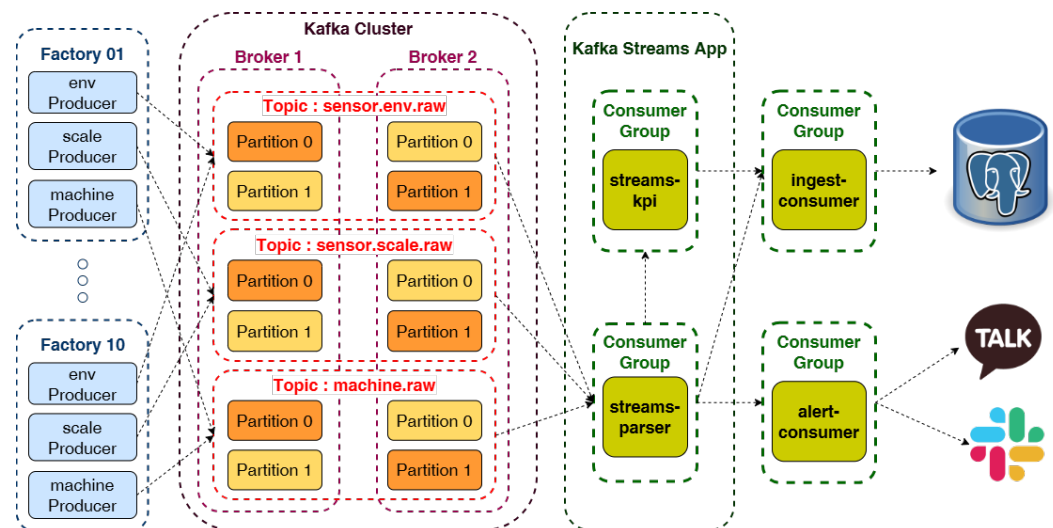
The factory-side acquisition layer was implemented as an edge-centered intake stack that received field data inside the plant, preserved them locally, and then forwarded them to the cloud event pipeline. Because the plant did not provide a uniform network-native source environment or stable plant-wide connectivity across distributed process areas, the architecture was organized around a local edge boundary rather than direct source-to-cloud transmission.

LoRaWAN was selected as the field communication network because it allowed distributed source collection across the plant without extensive wiring and supported locally controlled operation under constrained factory conditions. Accordingly, the plant-side network was organized around LoRa uplinks converging on a local gateway and edge node.

As shown in Figure 6, field events were transmitted through LoRa to the gateway, handled by ChirpStack Gateway Bridge and ChirpStack Server, relayed through an MQTT broker, and then accepted by edge-side services. On the edge node, MQTT logger and source-specific collectors wrote accepted records into local SQLite stores. This made the acquisition path explicit—field transmission, gateway reception, LoRaWAN handling, MQTT relay, edge logging, and upper-layer forwarding were treated as separate steps—and allowed the edge node to function as the first stable intake and persistence boundary of the system.

A central function of the edge stack was local persistence. Sensor uplinks accepted through MQTT were written to SQLite before cloud delivery was assumed, and additional edge-side components preserved operational traces needed to inspect collection quality and local failure modes. As a result, accepted field events first became durable inside the factory rather than remaining only as transient in-flight messages.

The edge stack also accommodated heterogeneous source conditions. A representative example was the production electronic scale, which did not provide native network communication. Its printer port was connected to the Raspberry Pi through an RS232 serial link, and a dedicated daemon collector parsed the incoming weighing values, stored them locally, and transmitted them through a LoRa device attached to the Raspberry Pi so that they could enter the same acquisition path as other field events. In practical terms, this



**Figure 7.** Enter Caption

flow can be summarized as *Scale* → *RS232* → *RPi* → *LoRa*. This showed that the edge architecture could absorb not only standard wireless sensor uplinks but also non-networked legacy equipment within the same factory-side event flow.

Once accepted and retained at the edge, the records were forwarded by producer services to the cloud broker layer. In the broader architecture, this upstream path was organized around Kafka-based streaming, including AWS MSK on the cloud side, so that raw events from each factory could be published into a shared event pipeline for subsequent parsing, normalization, and persistence. The next subsection explains that cloud-side processing path.

### 5.3. Cloud Processing Architecture

On top of the field acquisition layer, the system implemented a cloud-side processing architecture designed to interpret, derive, persist, and serve the collected data. The end-to-end flow connected edge producers, a message broker, parser streams, KPI streams, ingest consumers, a database layer, and downstream dashboard and alert services. Rather than concentrating all responsibilities in a single application, this architecture separated message intake, semantic normalization, derived computation, persistence, and service consumption into distinct functional stages.

A central design objective of this architecture was functional decoupling. Producer-side acquisition was not allowed to depend directly on parser execution, KPI computation speed, or database ingestion performance, because failures in downstream stages should not immediately halt the upstream intake path. For this reason, a brokered event flow was adopted to decouple acquisition from later processing. The architectural significance of the message broker was therefore not simply that Kafka was used, but that the acquisition path and the processing path were separated so that failure propagation could be reduced and downstream evolution could proceed more independently.

The parser stage consumed raw topics and performed decoding, source mapping, metric unification, timestamp handling, deduplication, and canonical event generation. Raw inputs such as *sensor.env.raw*, *sensor.scale.raw*, and *machine.raw* were transformed into a unified canonical stream. The function of the parser was not to forward device-specific payloads downstream, but to convert heterogeneous source messages into a common semantic form that later stages could consume without direct knowledge of device-specific structure.

After normalization, a dedicated KPI stream consumed canonical events and generated derived indicators such as production quantity and yield. KPI computation was intentionally separated from the parser stage and from query-time dashboard logic so that raw normalization and business-level derivation remained distinct responsibilities. This separation reduced semantic overloading in the parser, isolated derived computation from storage concerns, and preserved the possibility of replay under changed KPI rules.

The outputs of the parser and KPI streams were then delivered to the database through ingest consumers. These consumers were designed primarily as an append-only persistence path rather than as another transformation layer. In other words, semantic interpretation and derived computation remained in the stream layer, while the persistence layer focused on durable storage of already-governed events. This design preserved separation between processing logic and storage logic and supported later reprocessing and reuse.

The cloud architecture also maintained clear deployment boundaries. Shared event contracts were placed in a common module, while producers, parser streams, KPI streams, and ingest services were separated into unidirectional dependencies. Consumer-side components were kept lightweight and stateless where possible, while stateful processing responsibility remained with the broker and stream-processing layers. Taken together, the cloud stack was not merely a path for relaying raw events, but a governed, replay-aware, and service-ready processing architecture built on top of the continuity secured at the field edge.

**Table 3.** Cloud-side Components and Responsibilities.

| Component             | Input                        | Output                 | Primary responsibility                                          |
|-----------------------|------------------------------|------------------------|-----------------------------------------------------------------|
| Producer              | Edge-side records/messages   | Raw topics             | Forward field data to the cloud event flow                      |
| Parser stream         | Raw topics                   | Canonical events       | Decode, normalize, map sources, and standardize events          |
| KPI stream            | Canonical events             | KPI events             | Derive production- and process-level indicators                 |
| Ingest consumer       | Canonical events, KPI events | Database records       | Persist processed events without adding business logic          |
| Database              | Ingested records             | Queryable tables/views | Provide persistent storage for dashboards, alerts, and analysis |
| Dashboard/alert layer | Stored and derived data      | Operational interfaces | Present status, alarms, and management views                    |

5.4. Data Model

The data model was designed to provide stable semantics and replayable structure across heterogeneous sources. In a field environment where devices, payload formats, and process contexts differ substantially, exposing raw messages directly to downstream systems would have dispersed interpretation responsibility across the pipeline. For this reason, the implementation introduced a canonical event contract that internalized governance into the data path itself. The data model should therefore be understood not merely as a schema design choice, but as a structural mechanism for semantic control.



Each source was represented through a canonical identity organized around `source_id`, including attributes such as `tenant_id`, `line_id`, `process`, `device_type`, and `metric`. This identifier functioned as a semantic anchor rather than as a simple storage key. It allowed downstream components to operate on controlled source meaning without depending on raw device names or payload-specific fields, while a metadata layer preserved the source context in interpretable form.

Time semantics were also explicitly stabilized. In environments where sensor time, gateway time, network-server time, and receipt time may coexist, aggregation, deduplication, and persistence can become ambiguous if no common temporal reference is fixed. In this implementation, `edge_ingest_time` was used as the common pipeline reference time, while `event_time`, `edge_ingest_time`, and `db_ingest_time` were stored as distinct time axes with different roles. This separation prevented confusion between raw occurrence time, pipeline reference time, and durable persistence time, and supported consistent timestamp interpretation across the entire processing path.

Canonical events were generated not through simple field copying, but through normalization rules that included metric unification, value-type alignment, deduplication, and explicit missing-value handling. Deduplication was performed in the parser stream rather than in producers or at the database layer, using ordering and key-based logic appropriate to the message stream. At the same time, the implementation distinguished true zero values from missing observations and minimized unnecessary null propagation in canonical records. These rules were important not as isolated data-cleaning tricks, but as operational rules for preserving stable downstream meaning.

Storage was organized as a layered structure rather than as a single undifferentiated table space. Canonicalized records were written into raw-domain tables such as `raw.sensor_env`, `raw.sensor_scale`, and `raw.machine`; source interpretation metadata were stored in `meta.source`; and derived KPI events were stored in `derived.kpi`. This organization separated raw-like canonical persistence from business-level derivation and thus preserved the distinction between governed operational records and derived management logic.

The resulting model supported replayability and future change. Because the structure preserved a raw-to-canonical-to-derived chain, parser rules or KPI logic could be revised without collapsing the semantic basis of historical records. The combination of a common event contract, append-only ingest, and layered storage was therefore not a purely implementation-specific choice, but a structural condition for reprocessability and evolvability. At the same time, it provided the persistence basis on which dashboards, alerts, and other operational interfaces could be built.

### 5.5. Operations and Control

The implementation did not stop at data capture and storage. It was also connected to an operational layer through which plant operators and managers could observe state, receive alerts, and intervene when necessary. In this sense, the system was designed not as a hidden back-end pipeline, but as a data foundation that could be translated into operational visibility and action.

Dashboards provided real-time and cumulative views of temperature, humidity, weight, device status, battery condition, packet reception status, and derived KPIs. These views were not designed as raw-signal viewers alone, but as visual interfaces that reorganized operational data into line charts, status summaries, alert histories, and key figures suitable for day-to-day plant awareness. As a result, operators could inspect process conditions, environmental changes, sensor stability, and production-related indicators without relying solely on manual records.

The system also included an alert path for operational intervention. Threshold violations, prolonged silence from data sources, battery-related conditions, and other abnormal states were delivered as asynchronous alerts so that operators could respond promptly. Alert history was preserved alongside the dashboard layer, allowing visual monitoring and intervention support to remain linked rather than isolated. In this design, alerting was not treated as an accessory to monitoring, but as the path through which recognized states became actionable events.

Beyond immediate visibility, the processed and stored data also supported KPI tracking, reporting, and management-level review. Because raw-domain and derived layers were both preserved, the implementation supported not only real-time state checking, but also historical comparison, accumulation-based reporting, and line- or period-level operational review. The dashboard layer therefore served both operator-level situational awareness and manager-level visibility grounded in the same governed data basis.

The implementation further extended to machine-side control capability. The system was built to support actual power control of machinery in the field, meaning that it was capable not only of observing states and issuing alerts, but also of participating in operational intervention. This does not imply that the study centered on control automation itself; rather, it demonstrates that the operational data foundation had already reached the point where downstream device action could be connected to the same semantic and operational chain.

Overall, the operational layer linked dashboards, alerts, KPIs, and machine-side intervention on top of a single governed data foundation. This makes the implemented system more than a data collection pipeline: it functions as an operational interface through which perception, interpretation, response, and future application expansion can be organized on a common basis. The next section evaluates how this implementation satisfies the requirements and invariants defined in Section 4.

## 6. Evaluation of the Framework

This section evaluates whether the proposed framework functioned as an operational data foundation in a real manufacturing setting. The evaluation does not focus on conventional system benchmarking or algorithmic accuracy. Instead, it examines how the six framework requirements—continuity, governance, diagnosability, operability, reprocessability, and evolvability—became visible in the implemented system and in its operational history. Accordingly, Section 6.1 summarizes requirement-wise evidence, and Section 6.2 presents a representative field case in which continuity and diagnosability were most directly tested through the diagnosis and improvement of LoRa reception instability.

### 6.1. Framework Compliance

This subsection assesses the framework in terms of the six core requirements defined in Section 4. Rather than reducing each requirement to a single metric, the assessment draws on multiple forms of evidence, including source identity structure, time semantics, layered persistence, segment-wise observability, intervention history, and historical data retention. Table 4 summarizes the main evidence for each requirement and its significance, while the discussion below interprets these observations in operational terms.

**Table 4.** Requirement-wise Evidence and Significance.

| Requirement      | Key evidence                                               | Significance                        |
|------------------|------------------------------------------------------------|-------------------------------------|
| Continuity       | local capture, recovery path, reception improvement        | recoverable continuity              |
| Governance       | canonical schema, source identity, time semantics          | controlled and traceable meaning    |
| Diagnosability   | segment-wise metrics, gateway-to-DB discrepancy visibility | localizable and actionable failures |
| Operability      | deployable modular stack, monitoring/control interfaces    | manageable field operation          |
| Reprocessability | preserved raw data, layered storage                        | recomputation and correction        |
| Evolvability     | extensible source model, modular interfaces                | incremental expansion               |

**Continuity** was evaluated not as the perfect arrival of every event, but as the extent to which the event flow remained traceable and recoverable under unstable field conditions. In the implemented system, gateway reception, edge-side logging, downstream ingest, and database persistence could be observed as separate stages, allowing continuity to be interpreted through persisted-to-expected ratios, extended no-data intervals, and before–after improvement patterns. Under an expected daily volume of roughly 48,000 events, early persistence remained near 15,800 events, and some source groups exhibited repeated no-data gaps of 2–5 hours. After communication-related adjustments, persisted events recovered to approximately 45,600 per day, while extended gaps decreased from about 18 cases per day to fewer than two. These observations indicate that continuity in this system was not merely a matter of raw transmission success, but of maintaining a recoverable operational record despite intermediate instability.

**Governance** was reflected in whether heterogeneous field messages could be preserved and interpreted under a stable semantic discipline rather than being stored as loosely structured payloads. In the implemented system, source identity was organized around `source_id`, while the parser stage fixed metric normalization, timestamp normalization, deduplication, and missing-value handling before downstream use. The distinction among `event_time`, `edge_ingest_time`, and `db_ingest_time` prevented ambiguity between occurrence, pipeline reference, and persistence time. In addition, the separation of raw-like persistence from derived KPI layers preserved a stable semantic boundary between operational records and business-level interpretation. Taken together, these structures show that governance was realized as controlled meaning across collection, normalization, and storage.

**Diagnosability** was evaluated in terms of whether data loss or delay remained a black-box symptom or could be localized to specific lifecycle segments. Because gateway-visible counts, local logs, parser/consumer stages, and database persistence counts could be compared, the same symptom of “too little data” could be interpreted as distinct failure modes. In some intervals, gateway-visible counts were already well below expectations, directing attention to upstream collection and communication; in other situations, the discrepancy widened between gateway-visible counts and persisted records, indicating the need to inspect downstream processing or storage. This segment-wise visibility was especially important in the LoRa case discussed in Section 6.2, where the main source

of degradation could be narrowed to the communication-configuration layer rather than being misread as a parser or database issue.

**Operability** was reflected not merely in technical feasibility, but in whether the system could be deployed, inspected, tuned, and maintained under limited staffing and maintenance capacity. The implementation separated field-side acquisition, gateway and network services, edge persistence, cloud processing, database storage, and dashboard/alert interfaces by function, reducing unnecessary entanglement among acquisition, normalization, storage, derivation, and presentation. In actual operation, interventions such as rejoin handling, configuration adjustment, logging verification, interval tuning, and packet comparison could be performed at the relevant layer without redesigning the system as a whole. This indicates that the structure remained operationally manageable beyond a laboratory prototype and could be sustained under realistic factory conditions.

**Reprocessability** was assessed in terms of whether collected data remained available for future reinterpretation, recalculation, correction, and recovery rather than being consumed only by the current dashboard or rule logic. The implemented system avoided transformed-only persistence by separating raw-like event storage, source metadata, and derived KPI layers. As a result, historical records remained available for parser rule revision, KPI redefinition, historical audit, and retrospective reconstruction. The accumulated historical storage reached roughly 0.84–1.57 million rows, indicating that the system functioned not only as a short-term monitoring pipeline but also as a retained operational record base capable of supporting later reprocessing needs.

**Evolvability** was reflected in whether the structure could accept additional sources, process-specific measurements, monitoring tools, KPI logic, and downstream services without breaking its prevailing governance discipline. Because the system was organized around a common event contract, source identity discipline, parser-stage normalization, and modular responsibility separation, new sources could be incorporated primarily through onboarding rules and parser extensions rather than through full pipeline redesign. During the project, additional source classes, monitoring tools, derived KPI layers, and dashboard/alert functions were progressively introduced while the overall structure remained intact. This indicates that the framework supported staged expansion rather than one-time integration only.

Taken together, the evidence above shows that the proposed framework was realized not only as a conceptual structure but as an operational one. Among the six requirements, continuity and diagnosability were tested most directly through actual field instability, and their operational relevance becomes especially clear in the representative case discussed next.

## 6.2. Operational Case: LoRa Reliability Improvement

A field data foundation does not remain stable simply because it has once been installed. In practice, reception quality, device condition, battery state, wireless configuration, and operational usage patterns continue to change over time. For this reason, the value of the proposed framework lies not in whether it delivered perfect performance from the outset, but in whether it enabled emerging failures to be detected, localized, and corrected in an operationally meaningful way. This subsection presents a representative case in which severe reception degradation in periodic uplink collection was diagnosed and improved through the framework's continuity and diagnosability logic.

During the early operation period, the plant-wide event flow failed to maintain the expected level of periodic collection, and reception rates in some source groups dropped to roughly 33%. Under an expected daily volume of around 48,000 events, persisted events remained near 15,800, and repeated no-data intervals of 2–5 hours appeared in some groups.

This was not merely a matter of lower communication quality. It directly weakened line-level visibility, reduced confidence in dashboard and alert outputs, and undermined the continuity of the operational record.

Because the implemented system did not rely solely on final database status, this degradation could be interpreted as a segment-wise discrepancy rather than as a vague end-to-end symptom. Gateway-visible counts, local edge logs, parser/consumer stages, and database persistence counts could be examined together. In one representative 24-hour window, gateway-visible events were already only about 16,400, far below expectation, whereas the persistence ratio from gateway-visible events to database records remained around 97–99%. This pattern suggested that the main problem lay upstream of parsing and storage, making it more reasonable to investigate the collection and communication segment first.

Through this segment-wise diagnosis, the dominant cause was gradually localized not to parser failure or database loss, but to reception instability associated with the communication-configuration layer. In particular, the combination of confirmed mode, dependence on downlink acknowledgements, and limited channel availability was interpreted as a major source of degradation. In LoRa environments, confirmed uplinks require network-side downlink ACKs. When many field devices transmit at short intervals, downlink resources can become saturated, ACK delays or omissions accumulate, and retransmissions increase contention. Under such conditions, the visible symptom is uplink instability, but the underlying bottleneck is often downlink saturation. For example, if 60 field devices transmit every 5 minutes, the expected uplink load is about 720 messages per hour; even a 25% ACK miss rate can raise retransmission-induced traffic substantially and worsen congestion.

Once the likely bottleneck had been localized to the communication layer, the response proceeded through targeted intervention rather than unguided trial and error. The main adjustments included relaxation of confirmed-mode dependence, expansion of active channels, device rejoin, transmission-parameter tuning, and refinement of monitoring intervals. Each change corresponded to a specific bottleneck hypothesis, allowing recovery effects to be interpreted in relation to the observed loss pattern. The concrete configuration changes and their intended effects are summarized in Table 5.

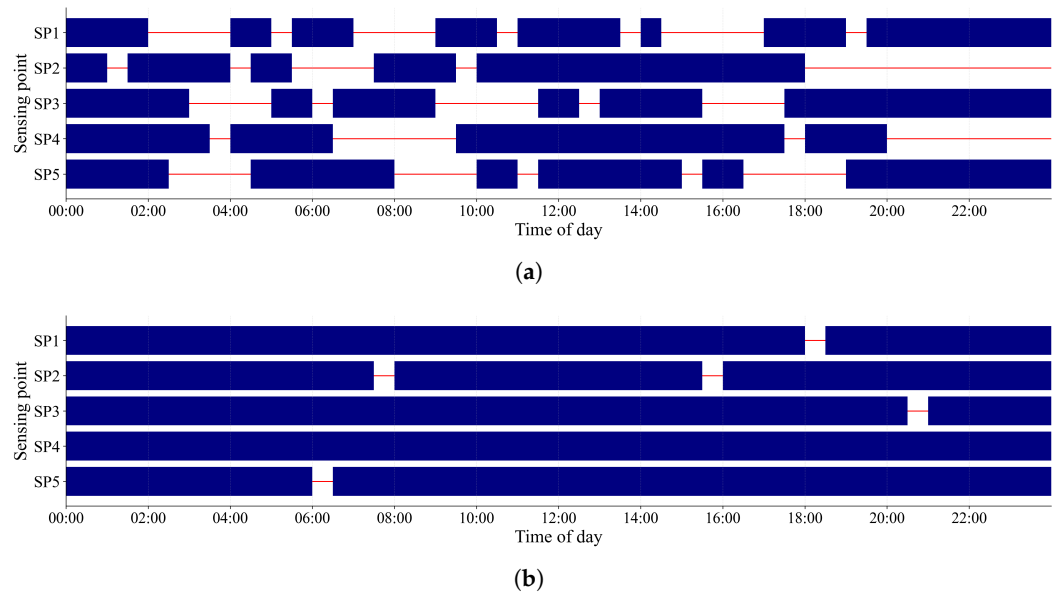
**Table 5.** Segment-wise Event Counts during Reception Degradation.

| Stage / Metric           | Expected | Observed | Ratio (%) |
|--------------------------|----------|----------|-----------|
| Scheduled transmissions  | 48,000   | –        | –         |
| Gateway-visible events   | 48,000   | 16,400   | 34.2      |
| Edge/local logged events | 16,400   | 16,120   | 98.3      |
| Ingested events          | 16,120   | 15,980   | 99.1      |
| DB-persisted events      | 15,980   | 15,870   | 99.3      |

After these adjustments, collection stability improved markedly. Packet reception increased from approximately 33% to 95%, daily persisted events rose from about 15,800 to 45,600, extended no-data intervals decreased from about 18 cases per day to fewer than two, and average gap duration dropped from roughly 145 minutes to 18 minutes. Time-series plots also showed that dense missing segments visible before adjustment were substantially reduced afterward, while observation regularity and line-to-line consistency improved (Figure ??; Table 6).



The significance of this case extends beyond successful LoRa tuning. It demonstrates that continuity degradation could be made observable, that the dominant cause could be narrowed to a specific lifecycle segment, and that the effects of intervention could be re-evaluated within the same operational structure. Continuity was revealed through degradation and recovery, diagnosability through segment-wise discrepancy visibility, and operability through configuration-based field intervention. Governance also mattered, because packet flow could be compared by source group and time window rather than only through aggregate counts. In this sense, the case shows that the proposed framework functioned not merely as a storage path, but as a problem-solving foundation that enabled unexpected field failures to be translated into operationally actionable improvement.



**Figure 8.** Reception stability across sensing points over time. (a) Before adjustment. (b) After adjustment.

**Table 6.** Outcome of Reception Reliability Improvement.

| Metric                        | Before | After  | Change  |
|-------------------------------|--------|--------|---------|
| Reception rate (%)            | 33     | 95     | +62 pp  |
| Persisted events/day          | 15,870 | 45,600 | +187%   |
| Average gap duration (min)    | 145    | 18     | −87.6%  |
| Long-gap events/day           | 18     | 2      | −88.9%  |
| Gateway-to-DB consistency (%) | 96.8   | 99.4   | +2.6 pp |

7. Conclusion

This study argued that the practical bottleneck of smart manufacturing in constrained SME factories lies not only in downstream analytics or intelligent applications, but more fundamentally in the absence of an operationally viable data foundation. In such environments, legacy equipment, fragmented systems, unstable field conditions, limited staffing, and cost sensitivity weaken the assumptions on which idealized smart manufacturing architectures are often built. The central question, therefore, is not simply how to use more

data, but how to design and sustain a data foundation that remains operationally viable under real deployment conditions.

To address this problem, this study proposed the *Operational Data Foundation Framework* for constrained SME factories. The framework formalized six core requirements—continuity, governance, diagnosability, operability, reprocessability, and evolvability—and connected them to corresponding design principles and operational invariants. In this way, the study contributed not merely a particular architecture, but a structured framework that links problem analysis, design logic, and assessment criteria for operational data foundations in smart manufacturing systems.

The framework was then instantiated in a real SME food-manufacturing deployment and evaluated from the perspective of operational viability rather than benchmark performance alone. Through requirement-based evidence and a representative field case, the study showed that the framework remained operationally meaningful in practice. In particular, the diagnosis and improvement of LoRa reception instability demonstrated that continuity and diagnosability were not abstract design goals, but directly relevant to the detection, interpretation, and correction of real operational problems.

More broadly, this study did not seek to add another downstream smart manufacturing application. Instead, it formalized the operational data foundation on which such applications depend. In this sense, the proposed framework provides a reference design logic for building data systems that can support real-time monitoring, traceability, analytics, alerting, and future extensions toward more adaptive and integrated manufacturing systems. Its contribution is therefore both practical and conceptual: it offers a field-grounded way to think about data-system design under constrained deployment conditions.

This study has several limitations. The framework was examined through a single SME food-manufacturing deployment, and its broader generalizability across multiple factories and more diverse industrial settings remains to be tested. In addition, although the implementation supported operational visibility, alerting, and machine-side intervention, the study did not directly evaluate large-scale multi-site deployment, fully automated replay workflows, or closed-loop control integration. Future work may therefore extend the framework through multi-factory replication, richer interoperability settings, automated regeneration and replay pipelines, and tighter integration with analytics, digital twins, and CPPS-oriented control.

**Author Contributions:** For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used “Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.”, please turn to the [CRediT taxonomy](#) for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

**Funding:** Please add: “This research received no external funding” or “This research was funded by NAME OF FUNDER grant number XXX.” and and “The APC was funded by XXX”. Check carefully that the details given are accurate and use the standard spelling of funding agency names at <https://search.crossref.org/funding>, any errors may affect your future funding.

**Data Availability Statement:** We encourage all authors of articles published in MDPI journals to share their research data. In this section, please provide details regarding where data supporting reported results can be found, including links to publicly archived datasets analyzed or generated during the study. Where no new data were created, or where data is unavailable due to privacy or ethical restrictions, a statement is still required. Suggested Data Availability Statements are available in section “MDPI Research Data Policies” at <https://www.mdpi.com/ethics>.

**Acknowledgments:** In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments). Where GenAI has been used for purposes such as generating text, data, or graphics, or for study design, data collection, analysis, or interpretation of data, please add “During the preparation of this manuscript/study, the author(s) used [tool name, version information] for the purposes of [description of use]. The authors have reviewed and edited the output and take full responsibility for the content of this publication.”

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Rozhok, A.; Abate, R.; Manoli, E.; Nele, L. A Review of Recent Advanced Applications in Smart Manufacturing Systems. *J. Manuf. Mater. Process.* **2026**, *10*, 1. <https://doi.org/10.3390/jmmp10010001>.
2. Bueno, A.; Godinho Filho, M.; Frank, A.G. Smart production planning and control in the Industry 4.0 context: A systematic literature review. *Comput. Ind. Eng.* **2020**, *149*, 106774. <https://doi.org/10.1016/j.cie.2020.106774>.
3. Cinar, Z.M.; Nuhu, A.A.; Zeeshan, Q.; Korhan, O. Digital Twins for Industry 4.0: A Review. In *Industrial Engineering in the Digital Disruption Era*; Calisir, F.; Korhan, O., Eds.; Lecture Notes in Management and Industrial Engineering, Springer: Cham, 2020. [https://doi.org/10.1007/978-3-030-42416-9\\_18](https://doi.org/10.1007/978-3-030-42416-9_18).
4. Atalay, M.; Murat, U.; Oksuz, B.; Parlaktuna, A.M.; Pisirir, E.; Testik, M.C. Digital twins in manufacturing: systematic literature review for physical–digital layer categorization and future research directions. *Int. J. Comput. Integr. Manuf.* **2022**, *35*, 679–705. <https://doi.org/10.1080/0951192X.2021.2022762>.
5. Song, Y. A Comprehensive Review of Key Technologies and Applications in Smart Manufacturing Systems: From Digital Foundations to Intelligent Applications. In Proceedings of the Proceedings of the 2025 2nd International Conference on Industrial Automation and Robotics, New York, NY, USA, 2025; IAR '25, pp. 328–333. <https://doi.org/10.1145/3778886.3778938>.
6. Nguyen, P.; Kim, M.; Nichols, E.; Yoon, H.S. AI-Driven Digital Twins for Manufacturing: A Review Across Hierarchical Manufacturing System Levels. *Sensors* **2025**, *26*, 124. <https://doi.org/10.3390/s26010124>.
7. Tao, F.; Zhang, M. Digital Twin Shop-Floor: A New Shop-Floor Paradigm Towards Smart Manufacturing. *IEEE Access* **2017**, *5*, 20418–20427. <https://doi.org/10.1109/ACCESS.2017.2756069>.
8. Ding, K.; Chan, F.T.S.; Zhang, X.; Zhou, G.; Zhang, F. Defining a Digital Twin-based Cyber-Physical Production System for autonomous manufacturing in smart shop floors. *Int. J. Prod. Res.* **2019**, *57*, 6315–6334. <https://doi.org/10.1080/00207543.2019.1566661>.
9. Ojstersek, R.; Javernik, A.; Buchmeister, B. Optimizing smart manufacturing systems using digital twin. *Adv. Prod. Eng. Manag.* **2023**, *18*, 475–485. <https://doi.org/10.14743/apem2023.4.486>.
10. Qamsane, Y.; Chen, C.Y.; Balta, E.C.; Kao, B.C.; Mohan, S.; Moyne, J.; Tilbury, D.; Barton, K. A Unified Digital Twin Framework for Real-time Monitoring and Evaluation of Smart Manufacturing Systems. In Proceedings of the 2019 IEEE 15th International Conference on Automation Science and Engineering (CASE), 2019, pp. 1394–1401. <https://doi.org/10.1109/COASE.2019.8843269>.
11. Ciano, M.P.; Pozzi, R.; Rossi, T.; Strozzi, F. Digital twin-enabled smart industrial systems: a bibliometric review. *Int. J. Comput. Integr. Manuf.* **2021**, *34*, 690–708. <https://doi.org/10.1080/0951192X.2020.1852600>.
12. Ismail, A.; Truong, H.L.; Kastner, W. Manufacturing process data analysis pipelines: a requirements analysis and survey. *J. Big Data* **2019**, *6*, 1. <https://doi.org/10.1186/s40537-018-0162-3>.
13. Raptis, T.P.; Passarella, A.; Conti, M. Data Management in Industry 4.0: State of the Art and Open Challenges. *IEEE Access* **2019**, *7*, 97052–97093. <https://doi.org/10.1109/ACCESS.2019.2929296>.
14. Saqlain, M.; Piao, M.; Shim, Y.; Lee, J.Y. Framework of an IoT-based Industrial Data Management for Smart Manufacturing. *J. Sens. Actuator Netw.* **2019**, *8*, 25. <https://doi.org/10.3390/jsan8020025>.
15. O'Donovan, P.; Leahy, K.; Bruton, K.; O'Sullivan, D.T.J. An industrial big data pipeline for data-driven analytics maintenance applications in large-scale smart manufacturing facilities. *J. Big Data* **2015**, *2*, 25. <https://doi.org/10.1186/s40537-015-0034-z>.
16. Zeid, A.; Sundaram, S.; Moghaddam, M.; Kamarthi, S.; Marion, T. Interoperability in Smart Manufacturing: Research Challenges. *Machines* **2019**, *7*, 21. <https://doi.org/10.3390/machines7020021>.
17. Pereira, R.M.; Szejka, A.L.; Cancigliieri Junior, O. Towards an information semantic interoperability in smart manufacturing systems: contributions, limitations and applications. *Int. J. Comput. Integr. Manuf.* **2021**, *34*, 422–439. <https://doi.org/10.1080/0951192X.2021.1891571>.
18. Modoni, G.E.; Doukas, M.; Terkaj, W.; Sacco, M.; Mourtzis, D. Enhancing factory data integration through the development of an ontology: from the reference models reuse to the semantic conversion of the legacy models. *Int. J. Comput. Integr. Manuf.* **2017**, *30*, 1043–1059. <https://doi.org/10.1080/0951192X.2016.1268720>.

19. Hildebrandt, C.; Köcher, A.; Küstner, C.; López-Enríquez, C.M.; Müller, A.W.; Caesar, B.; Gundlach, C.S.; Fay, A. Ontology Building for Cyber–Physical Systems: Application in the Manufacturing Domain. *IEEE Trans. Autom. Sci. Eng.* **2020**, *17*, 1266–1282. <https://doi.org/10.1109/TASE.2020.2991777>. 1011
20. Morris, K.C.; Lu, Y.; Frechette, S. Foundations of Information Governance for Smart Manufacturing. *Smart Sustain. Manuf. Syst.* **2020**, *4*, 43–61. <https://doi.org/10.1520/SSMS20190041>. 1012
21. Westermann, T.; Hranisavljevic, N.; Fay, A. Accessing and Interpreting OPC UA Event Traces based on Semantic Process Descriptions. In Proceedings of the 2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA), Stuttgart, Germany, 2022; pp. 1–7. <https://doi.org/10.1109/ETFA52439.2022.9921565>. 1013
22. Abdel-Aty, T.A.; Negri, E. Conceptualizing the digital thread for smart manufacturing: a systematic literature review. *J. Intell. Manuf.* **2024**, *35*, 3629–3653. <https://doi.org/10.1007/s10845-024-02407-1>. 1014
23. Zhang, Q.; Liu, J.; Chen, X. A Literature Review of the Digital Thread: Definition, Key Technologies, and Applications. *Systems* **2024**, *12*, 70. <https://doi.org/10.3390/systems12030070>. 1015
24. Bianchini, D.; Fapanni, T.; Garda, M.; Leotta, F.; Mecella, M.; Rula, A.; Sardini, E. Digital Thread for Smart Products: A Survey on Technologies, Challenges, and Opportunities in Service-Oriented Supply Chains. *IEEE Access* **2024**, *12*, 125284–125305. <https://doi.org/10.1109/ACCESS.2024.3454375>. 1016
25. Hedberg Jr., T.D.; Bajaj, M.; Camelio, J.A. Using Graphs to Link Data Across the Product Lifecycle for Enabling Smart Manufacturing Digital Threads. *J. Comput. Inf. Sci. Eng.* **2020**, *20*, 011011. <https://doi.org/10.1115/1.4044921>. 1017
26. Kwon, S.; Monnier, L.V.; Barbau, R.; Bernstein, W.Z. Enriching standards-based digital thread by fusing as-designed and as-inspected data using knowledge graphs. *Adv. Eng. Inform.* **2020**, *46*, 101102. <https://doi.org/10.1016/j.aei.2020.101102>. 1018
27. Schmidt, N.; Lueder, A. The Flow and Reuse of Data: Capabilities of AutomationML in the Production System Life Cycle. *IEEE Ind. Electron. Mag.* **2018**, *12*, 59–63. <https://doi.org/10.1109/MIE.2018.2818748>. 1019
28. Monnier, L.V.; Shao, G.; Fofou, S. A Methodology for Digital Twins of Product Lifecycle Supported by Digital Thread. In Proceedings of the Proceedings of the ASME 2022 International Mechanical Engineering Congress and Exposition, Columbus, Ohio, USA, 2022; p. V02BT02A023. <https://doi.org/10.1115/IMECE2022-95182>. 1020
29. Mittal, S.; Khan, M.A.; Romero, D.; Wuest, T. A critical review of smart manufacturing & Industry 4.0 maturity models: Implications for small and medium-sized enterprises (SMEs). *J. Manuf. Syst.* **2018**, *49*, 194–214. <https://doi.org/10.1016/j.jmsy.2018.10.005>. 1021
30. Mittal, S.; Khan, M.A.; Purohit, J.K.; Menon, K.; Romero, D.; Wuest, T. A smart manufacturing adoption framework for SMEs. *Int. J. Prod. Res.* **2020**, *58*, 1555–1573. <https://doi.org/10.1080/00207543.2019.1661540>. 1022
31. Ghobakhloo, M.; Ching, N.T. Adoption of digital technologies of smart manufacturing in SMEs. *J. Ind. Inf. Integr.* **2019**, *16*, 100107. <https://doi.org/10.1016/j.jii.2019.100107>. 1023
32. Krishnan, R. Challenges and benefits for small and medium enterprises in the transformation to smart manufacturing: a systematic literature review and framework. *J. Manuf. Technol. Manag.* **2024**, *35*, 918–938. <https://doi.org/10.1108/JMTM-07-2022-0255>. 1024
33. Alqoud, A.; Schaefer, D.; Milisavljevic-Syed, J. Industry 4.0: a systematic review of legacy manufacturing system digital retrofitting. *Manufacturing Rev.* **2022**, *9*, 32. [https://doi.org/10.1007/978-3-030-57997-5\\_13](https://doi.org/10.1007/978-3-030-57997-5_13). 1025
34. Etz, D.; Brantner, H.; Kastner, W. Smart Manufacturing Retrofit for Brownfield Systems. *Procedia Manuf.* **2020**, *42*, 327–332. <https://doi.org/10.1016/j.promfg.2020.02.085>. 1026
35. Tran, T.A.; Ruppert, T.; Eigner, G.; Abonyi, J. Retrofitting-Based Development of Brownfield Industry 4.0 and Industry 5.0 Solutions. *IEEE Access* **2022**, *10*, 64348–64374. <https://doi.org/10.1109/ACCESS.2022.3182491>. 1027
36. Park, H.M.; Jeon, J.W. OPC UA based Universal Edge Gateway for Legacy Equipment. In Proceedings of the 2019 IEEE 17th International Conference on Industrial Informatics (INDIN), Helsinki, Finland, 2019; pp. 1002–1007. <https://doi.org/10.1109/INDIN41052.2019.8972187>. 1028
37. Huang, D.; Chin, C.P.Y. The Barriers and Challenges in Smart Manufacturing Adoption for SMEs: A Review. *J. Adv. Manuf. Syst.* **2025**, *24*, 533–557. <https://doi.org/10.1142/S0219686725500246>. 1029
38. Narwane, V.S.; Raut, R.D.; Gardas, B.B.; Narkhede, B.E.; Awasthi, A. Examining smart manufacturing challenges in the context of micro, small and medium enterprises. *Int. J. Comput. Integr. Manuf.* **2022**, *35*, 1395–1412. <https://doi.org/10.1080/0951192X.2022.2078508>. 1030
39. Kumar, R.; Dutta, G.; Phanden, R.K. Digitalization Adoption Barriers in the Context of Sustainability and Operational Excellence: Implications for SMEs. *Eng. Manag. J.* **2025**, *37*, 355–371. <https://doi.org/10.1080/10429247.2024.2372519>. 1031
40. Hulla, M.; Herstätter, P.; Wolf, M.; Ramsauer, C. Towards digitalization in production in SMEs – A qualitative study of challenges, competencies and requirements for trainings. *Procedia CIRP* **2021**, *104*, 887–892. <https://doi.org/10.1016/j.procir.2021.11.149>. 1032
41. Chavez, Z.; Baalsrud Hauge, J.; Bellgran, M. A Conceptual Model for Deploying Digitalization in SMEs Through Capability Building. In *Advances in Production Management Systems. Towards Smart and Digital Manufacturing*; Lalic, B.; Majstorovic, V.; Marjanovic, U.; von Cieminski, G.; Romero, D., Eds.; Springer: Cham, 2020; Vol. 592, *IFIP Advances in Information and Communication Technology*. [https://doi.org/10.1007/978-3-030-57997-5\\_13](https://doi.org/10.1007/978-3-030-57997-5_13). 1033

42. Banerjee, A.; Jayaraman, P.P.; Fizza, K.; Wang, S.; Jin, J.; Ghaderi, H. Low-cost digital manufacturing solution for process manufacturing SMEs - Lesson and experiences from real-world pilot. *IET Conf. Proc.* **2024**, *2024*, 109–115. <https://doi.org/10.1049/icp.2024.3494>. 1066
43. Foidl, H.; Golendukhina, V.; Ramler, R.; Felderer, M. Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *J. Syst. Softw.* **2024**, *207*, 111855. <https://doi.org/10.1016/j.jss.2023.111855>. 1067
44. Rauch, E.; Dallasega, P.; Unterhofer, M. Requirements and Barriers for Introducing Smart Manufacturing in Small and Medium-Sized Enterprises. *IEEE Eng. Manag. Rev.* **2019**, *47*, 87–94. <https://doi.org/10.1109/EMR.2019.2931564>. 1068
45. Gao, C.; Wang, Z.; Chen, Y. On the Connectivity of Highly Dynamic Wireless Sensor Networks in Smart Factory. In Proceedings of the 2019 International Conference on Networking and Network Applications (NaNA), Daegu, Korea (South), 2019; pp. 208–212. <https://doi.org/10.1109/NaNA.2019.00045>. 1069
46. Noor-A-Rahim, M.; John, J.; Firyaguna, F.; Sherazi, H.H.R.; Kushch, S.; Vijayan, A.; O'Connell, E.; Pesch, D.; O'Flynn, B.; O'Brien, W.; et al. Wireless Communications for Smart Manufacturing and Industrial IoT: Existing Technologies, 5G and Beyond. *Sensors* **2023**, *23*, 73. <https://doi.org/10.3390/s23010073>. 1070
47. Hawkrigde, G.; Hernandez, M.P.; de Silva, L.; Terrazas, G.; Tlegenov, Y.; McFarlane, D.; Thorne, A. Tying Together Solutions for Digital Manufacturing: Assessment of Connectivity Technologies & Approaches. In Proceedings of the 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), Zaragoza, Spain, 2019; pp. 1383–1387. <https://doi.org/10.1109/ETFA.2019.8869411>. 1071
48. Nagorny, K.; Scholze, S.; Colombo, A.W.; Oliveira, J.B. A DIN Spec 91345 RAMI 4.0 Compliant Data Pipelining Model: An Approach to Support Data Understanding and Data Acquisition in Smart Manufacturing Environments. *IEEE Access* **2020**, *8*, 223114–223129. <https://doi.org/10.1109/ACCESS.2020.3045111>. 1072
49. Nasirinejad, M.; Afshari, H.; Sampalli, S. Challenges and Solutions to Adopt Smart Maintenance in SMEs: A Literature Review and Research Agenda. *IFAC-PapersOnLine* **2024**, *58*, 917–922. <https://doi.org/10.1016/j.ifacol.2024.09.164>. 1073
50. Doyle, F.; Cosgrove, J. Steps towards digitization of manufacturing in an SME environment. *Procedia Manuf.* **2019**, *38*, 540–547. <https://doi.org/10.1016/j.promfg.2020.01.068>. 1074
51. McFarlane, D.; Ratchev, S.; de Silva, L.; Hawkrigde, G.; Schönfuß, B.; Terrazas Angulo, G. Digitalisation for SME Manufacturers: A Framework and a Low-Cost Approach. *IFAC-PapersOnLine* **2022**, *55*, 414–419. <https://doi.org/10.1016/j.ifacol.2022.04.229>. 1075

**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 1092